

Claude e Claude Code in pratica

Dal prompting agli agenti

Simone Scannapieco

Corso base per Tecnodata Trentina S.r.l., Trento, Italia

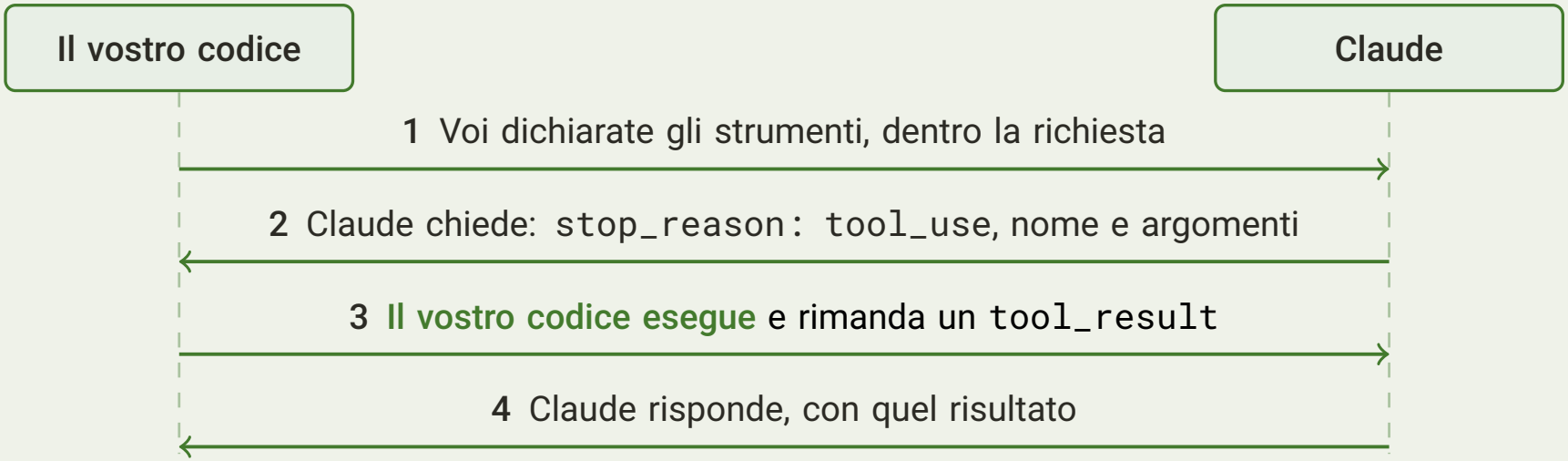
Modulo 7 — Anatomia, conversazione e strumenti

Strumenti: il modello chiede, il codice esegue

Il modello non esegue nulla

Tool use: quattro mosse, e la terza è vostra

Un cliente di CiaspoLink, il fornitore di connettività inventato del corso, chiede come sta la sua linea: il dato non sta nel modello, sta nei sistemi del fornitore. Uno **strumento** (*tool*) è il modo in cui l'assistente lo chiede, in quattro mosse



La terza mossa è **vostra**: il modello non tocca i vostri sistemi. Chiede, e il vostro codice decide se eseguire, come e con quali permessi

La documentazione di Anthropic: «The model never runs anything on its own. It emits a structured request, your code (or Anthropic's servers) runs the operation»

Dichiarare uno strumento

Nome, descrizione, parametri: la descrizione è il contratto

M7-D03, a capo per la slide, senza un campo del banco

```
{
  "name": "stato_linea",
  "description": "Legge lo stato attuale di una linea di
    CiaspoLink dal sistema di rete: stato, tecnologia, offerta,
    dettaglio dell'eventuale guasto ed eventuale guasto collettivo
    in zona. Usalo ogni volta che il cliente chiede come sta una
    linea o perché non funziona; non usarlo per dati di
    fatturazione. Il parametro id_linea è il codice della linea
    nel formato LN-000000. Se il codice non esiste lo strumento
    restituisce un errore: in quel caso chiedi al cliente di
    ricontrollarlo.",
  "input_schema": {"type": "object",
    "properties": {"id_linea": {"type": "string", "description":
      "Il codice della linea, nel formato LN-000000 (per esempio
      LN-000201)."}},
    "required": ["id_linea"]}
}
```

Dichiarare uno strumento

Nome, descrizione, parametri: la descrizione è il contratto

- Lo strumento della demo che segue, nel formato dell'API: tre campi, `name`, `description` e `input_schema`
- Dalla guida alla definizione degli strumenti: «Provide extremely detailed descriptions. This is by far the most important factor in tool performance.» Che cosa deve dire, letto sulla nostra:

da dire, per Anthropic	nella descrizione di `stato_linea`
che cosa fa lo strumento	«Legge lo stato attuale di una linea di CiaspoLink dal sistema di rete […]»
quando usarlo, e quando no	«Usalo ogni volta che il cliente chiede come sta una linea o perché non funziona; non usarlo per dati di fatturazione.»
che cosa significa ogni parametro	«Il parametro id_linea è il codice della linea nel formato LN-000000.»
come si comporta	«Se il codice non esiste lo strumento restituisce un errore: in quel caso chiedi al cliente di ricontrollarlo.»

- Almeno **3–4 frasi**, di più se lo strumento è complesso: qui sono quattro. `input_schema` è un oggetto JSON Schema con i parametri attesi
- «The description is the contract. It's the only thing Claude reads before deciding whether to call the tool.», dalla guida di Anthropic ai server MCP

M7-D03 Il modello chiede, il codice risponde

Demo del docente in Claude Code: un server MCP locale al posto del sistema vero

- Due frasi da mettere alla prova. Dalla documentazione: «The model never runs anything on its own. It emits a structured request, your code (or Anthropic's servers) runs the operation». Dalla pagina su MCP di Claude Code: «when a tool returns a result marked isError: true, Claude receives the result's text as the tool's error message.»
- L'assistente di CiaspoLink ha un solo strumento, `stato_linea`, e un **server MCP locale** con dati sintetici al posto del sistema vero. Una chiamata di `claude -p` per messaggio, Sonnet ed effort low:

```
i due messaggi, testi sotto prova, a capo per la slide

Buongiorno, ho la linea LN-000201 e non ho più internet. Mi dite
come sta la mia linea?
Buongiorno, ho la linea LN-999999 e non ho più internet. Mi dite
come sta la mia linea?
```

```
# dalla cartella esercitazioni
m7_assistenti_llm/s5_anatomia_e_strumenti/m7_5_strumenti/\
M7-D03_il_codice_risponde/demo/in_classe.sh \
  ~/corso-claude-code/M7-D03
```

- Guardate il **registro del server**: ogni chiamata che il modello chiede deve stare lì, con gli stessi argomenti; per la linea che non esiste, il risultato deve arrivare a Claude come **errore**

M7-D03 Il modello chiede, il codice risponde

Demo del docente in Claude Code: un server MCP locale al posto del sistema vero

La registrazione del corso, tre esecuzioni dei due casi. Sulla **linea che esiste**, in tutte e tre: una chiamata, uguale nello stream e nel registro del server, e la risposta dopo il risultato. Sulla **linea che non esiste**, la prima:

```
run-1, le righe del controllo: «[…]» segna un taglio

run-1, linea-inesistente
 il modello ha chiesto (stream), il server ha risposto (registro):
    stato_linea, argomenti {"id_linea": "LN-999999"}
      risultato: {"errore": "linea non trovata: serve il codice
        LN-000000"}
      nel tool_result: is_error true
  […]
  «The model never runs anything on its own»: confermata
  «Claude receives the result's text as the tool's error message»:
    nel registro errore sì, nel tool_result is_error true:
    confermata
```

«Le due garanzie, esecuzione per esecuzione: valgono in 3 su 3.» La prima **confermata** in tutti e sei i casi, la seconda nei tre della linea che non esiste. Dimostrazione dello script, non misura su Claude: nella risposta sulla linea che esiste, nessun numero, codice o orario assente dal risultato

Registrazione del corso, 9 novembre 2026, Claude Code 2.1.319, Sonnet, effort low

Quattro mestieri di uno strumento

Che cosa fa fare uno strumento a un assistente, e quanti ne servono

- **Lookup**: un dato preciso, per codice. Sul banco di prova del corso, l'assistente simulato delle esercitazioni, `stato_linea`
- **Ricerca**: poche righe pertinenti invece di tutto. Sul banco, `cerca_kb` nelle procedure; l'esempio di Anthropic: invece di un `read_logs`, un `search_logs` che restituisce solo le righe pertinenti con un po' di contesto
- **Dati strutturati**: campi con una forma garantita, come una fattura letta per voci. Sul banco, `leggi_fattura`
- **Azioni che cambiano lo stato**: un accredito, un intervento prenotato: sul banco `registra_accredito` e `prenota_intervento`. Dopo una modifica lo strumento restituisce una conferma breve e gli identificativi che servono alle chiamate dopo
- Per Anthropic il tool use serve proprio a questo: azioni con effetti collaterali, dati freschi o esterni, output con una forma garantita, chiamate a sistemi esistenti

Quattro mestieri di uno strumento

Che cosa fa fare uno strumento a un assistente, e quanti ne servono

- Quanti: «More tools don’t always lead to better outcomes.» Un errore comune è incapsulare soltanto funzioni o endpoint esistenti; Anthropic raccomanda **pochi strumenti ragionati** su flussi specifici ad alto impatto, e di crescere da lì
- Il suo esempio: invece di `get_customer_by_id`, `list_transactions` e `list_notes`, un solo `get_customer_context` che raccolga le informazioni recenti e pertinenti del cliente
- *Poka-yoke*: cambiare gli argomenti in modo che sbagliare sia più difficile
- Su lettura e scrittura due pagine di Anthropic **non concordano**: la guida alle definizioni consiglia di raggruppare le azioni correlate in un solo strumento con un parametro `action`; i criteri della directory dei connettori chiedono di dividere uno strumento *catch-all* in uno di sola lettura e uno o più di scrittura. Nel laboratorio si riprogetta e si misura

M7-D04 La descrizione è il contratto

Demo del docente in Claude Code: cambiano solo le definizioni

> La raccomandazione da mettere alla prova, dalla guida alla definizione degli strumenti: «Provide extremely detailed descriptions. This is by far the most important factor in tool performance.»

Stesso system prompt, stessi dati. La cliente: «Buongiorno, sono Laura Prova, codice cliente CL-000101, e-mail laura.prova@example.org. […]» e chiede della sua linea. **Cambiano solo le definizioni**:

capacità	definizioni vaghe	definizioni chiare
identità	`check_cliente`: «Controlla il cliente.»	`verifica_identita`: «[…] Va chiamato per primo, in ogni conversazione […]»
scheda	`cercaCliente`: «Cerca un cliente […] e restituisce la scheda completa […]»	`leggi_scheda_cliente`: «[…] solo dopo che verifica_identita ha dato esito “riconosciuto” […]»
linea	`check_linea`: «Controlla la linea del cliente.»	`stato_linea`: «[…] solo dopo che verifica_identita ha dato esito “riconosciuto” […]»

```
# dalla cartella esercitazioni
m7_assistenti_llm/s5_anatomia_e_strumenti/m7_5_strumenti/\
M7-D04_la_descrizione/demo/in_classe.sh \
  ~/corso-claude-code/M7-D04
```

Guardate il primo strumento chiamato e i suoi argomenti. **Indovinate, a voce**: con la definizione vaga, quale strumento chiamerà per primo?

M7-D04 La descrizione è il contratto

Demo del docente in Claude Code: cambiano solo le definizioni

La registrazione del corso, tre esecuzioni per versione. La risposta: con le definizioni vaghe, `cercaCliente`, in tutte e tre

	definizioni vaghe	definizioni chiare
primo strumento	`cercaCliente`, 3 su 3	`verifica_identita`, 3 su 3
poi	`check_linea`, 2 su 3	`stato_linea`, 3 su 3
verifica d'identità prima dei dati	no, 3 su 3	sì, 3 su 3
proprietà rispettate, delle tre	1 su 3, ogni esecuzione	3 su 3, ogni esecuzione
token della prima richiesta	1.481–1.482	2.287–2.288

```
la coppia, come il controllo la stampa
```

```
esecuzione 1: vaga 1 su 3, chiara 3 su 3: confermata
esecuzione 2: vaga 1 su 3, chiara 3 su 3: confermata
esecuzione 3: vaga 1 su 3, chiara 3 su 3: confermata
confermata 3 su 3
```

Le tre proprietà: il primo strumento è la verifica d'identità; la verifica precede le letture dei dati; gli argomenti nel formato atteso. Con le vaghe la scheda completa si legge **senza verifica d'identità**. Una raccomandazione: tre esecuzioni dicono che cosa è successo **qui**, non una regola generale

Registrazione del corso, 9 novembre 2026, Claude Code 2.1.319, Sonnet, effort low

Le azioni che cambiano lo stato

Il modello propone, una persona o una regola applica

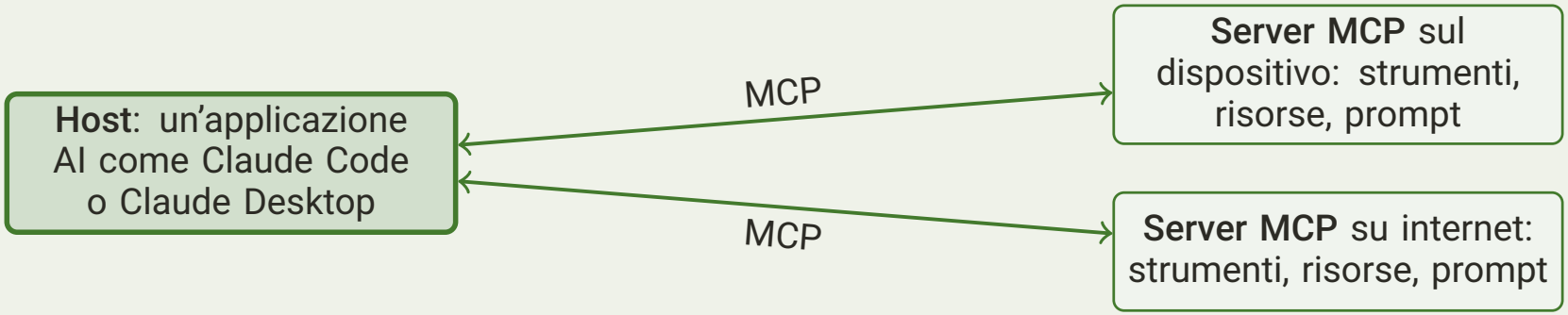
- Per Anthropic le persone devono mantenere il **controllo** su come si perseguono i loro obiettivi, soprattutto prima di decisioni ad alto rischio
- Il suo esempio: si può decidere che leggere il calendario sia sempre sicuro, e chiedere comunque l'**approvazione** prima di inviare un invito. Per CiaspoLink: leggere lo stato di una linea, e registrare un accredito
- Sugli agenti per il commercio Anthropic scrive che «the model's most dangerous action is to **propose**»: ad applicare sono una persona o una policy. Nel suo blueprint ogni scrittura proposta è una **modifica in attesa**, con un identificativo del server, che l'operatore vede in anteprima, e si applica solo dopo l'approvazione di una persona fuori dalla conversazione: «An approval typed in chat approves nothing.»
- Sul banco di prova è un'asserzione: nei cinque casi, anche davanti al cliente furioso o all'istruzione nascosta, nessuna chiamata allo strumento dell'accredito

La descrizione è il contratto, e il codice decide che cosa è permesso.

MCP: perché un protocollo

Uno standard aperto per collegare le applicazioni AI a strumenti e dati

- Senza uno standard, ogni applicazione scrive e mantiene la sua integrazione per ogni servizio: **N servizi, N integrazioni**
- Il 25 novembre 2024 Anthropic ha reso open source il **Model Context Protocol**, «a new standard for connecting AI assistants to the systems where data lives»; poi l'ha donato all'Agentic AI Foundation, un fondo della Linux Foundation, dichiarando oltre 10.000 server pubblici attivi e l'adozione di ChatGPT, Cursor, Gemini, Microsoft Copilot, Visual Studio Code e altri prodotti



- Un **host** si collega a uno o più server MCP; il server gira sul dispositivo dell'utente o su internet. In locale, sul trasporto stdio, il client lancia il server come sottoprocesso e comunica su input e output standard: è il server della demo M7-D03
- Un server espone tre cose: gli **strumenti** (azioni che Claude può compiere), che controlla Claude; le **risorse** (dati a cui può accedere), che controlla il codice dell'applicazione; i **prompt** (interazioni predefinite per compiti specifici), che avvia l'utente

Tool use o server MCP?

Non si escludono: dove sta lo strumento, e chi lo usa

Sono complementari: «MCP servers provide tool schemas and functions already defined for you, while tool use is about how Claude actually calls those tools.»

	Strumento nell'applicazione	Server MCP
chi lo definisce	la vostra applicazione, nella richiesta	il server, che fornisce schemi e funzioni già definiti
chi lo chiama	Claude, con il tool use	Claude, con il tool use
quante applicazioni lo usano	una, quella che lo contiene	tutte quelle che si collegano al server
che cosa offre	strumenti	strumenti, risorse e prompt
dove gira	con la vostra applicazione	locale, per provare da Claude Code o Claude Desktop; remoto per claude.ai (connettori) e per l'API (server esposto in HTTP, non stdio)

I server locali configurati in Claude Desktop sono un meccanismo separato: non sono disponibili in Cowork o su claude.ai